Київський національний університет імені
Тараса Шевченка
Факультет комп’ютерних наук та кібернетики

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ

з дисципліни “Розподілене та паралельне програмування”

на тему: Паралельна кластеризація текстових документів (TF-IDF + K-Means) у системі ПАРКС у хмарному середовищі Google Cloud Platform

Студента 3 курсу
групи ТК-31
спеціальності 122 “Комп’ютерні науки”
Ладіков-Роєв Микола Олександрович

Київ 2026

Мета роботи

Обрати задачу паралельного програмування (обробка текстових даних — кластеризація текстів) та реалізувати її для платформи ПАРКС (PARCS-python). Розгорнути систему ПАРКС у хмарному середовищі Google Cloud Platform, провести тестування послідовного та паралельного алгоритмів для різної кількості воркерів, порівняти результати та оцінити прискорення й ефективність паралельної програми.

Вступ

Кластеризація текстів — це задача навчання без учителя: множину документів потрібно розбити на групи так, щоб документи однієї групи були схожими за змістом, а різних — відрізнялися. Вона застосовується для організації великих колекцій документів, тематичного аналізу, побудови пошукових і рекомендаційних систем. Класичний підхід — подати кожен документ вектором TF-IDF і застосувати алгоритм K-Means.

K-Means є ітеративним, і основну частину часу на кожній ітерації займає обчислення відстаней між кожним документом і кожним центроїдом. Відстані для різних документів обчислюються незалежно, а підсумовування за кластерами є асоціативною операцією. Тому задача природно розпаралелюється за даними (data parallelism) за схемою Map-Reduce.

ПАРКС (Паралельні Асинхронні Рекурсивно Керовані Системи) — технологія й система для паралельних обчислень у комп’ютерних мережах, що ґрунтується на концепції керуючого простору. У роботі використано реалізацію PARCS-python: вузол master приймає завдання через веб-інтерфейс і розподіляє роботу між вузлами-воркерами, які викликаються віддалено через Pyro4. Кожен вузол розгортається як Docker-контейнер на віртуальній машині Google Compute Engine.

1. Алгоритм кластеризації

1.1. Векторне представлення тексту (TF-IDF)

Документ перетворюється на вектор у кілька кроків. Спочатку виконується токенізація: текст переводиться в нижній регістр, залишаються слова з латинських літер довжиною щонайменше три символи, стоп-слова відкидаються. Словник складається з 1000 слів, які трапляються щонайменше у двох документах і не більше ніж у 50 % документів, відібраних за спаданням частоти. Вага слова t у документі d дорівнює

$$w(t, d) = (1 + \ln \mathrm{tf}(t, d)) \cdot \mathrm{idf}(t), \quad \mathrm{idf}(t) = \ln((1 + N) / (1 + \mathrm{df}(t))) + 1,$$

де N — кількість документів, df(t) — кількість документів зі словом t. Кожен вектор нормується за нормою L2, а значення округлюються до шести значущих цифр (однаково для послідовної й паралельної версій).

1.2. Сферичний K-Means

Оскільки вектори нормовані, схожість документів вимірюється косинусом кута між ними. Квадрат відстані між документом x ($\|x\| = 1$) і центроїдом c обчислюється як $d(x, c) = 1 - 2 \cdot (x \cdot c) + \|c\|^2$. Алгоритм виконується так:

- початкові K центроїдів — випадкові документи (фіксований seed = 42, тому обидві версії стартують однаково);
- призначення: кожен документ відносять до кластера з найменшою відстанню d(x, c);

- оновлення: новий центроїд кластера — нормована сума векторів його документів, c_j = Σx / ‖Σx‖;
- повторення кроків 2–3 задану кількість ітерацій (або доки відносна зміна inertia = Σ min d не стане меншою за tol).

Складність однієї ітерації становить O(N · z · K), де z — середня кількість ненульових компонент вектора документа. Для K = 20 це основна обчислювальна вартість, яку й розподіляють між воркерами.

2. Принцип роботи та розпаралелювання

Програма реалізує схему Map-Reduce, що повторюється на кожній ітерації K-Means. Архітектура включає такі етапи:

- **Підготовка (master, послідовно).** Master читає вхідний файл, будує TF-IDF-вектори та початкові центроїди. Це послідовна частина алгоритму.
- **Декомпозиція.** N документів ділиться на W рівних частин (чанків) за кількістю воркерів; якщо N не ділиться націло, останній чанк менший.
- **Map (воркер).** Метод assign_chunk отримує свій чанк і поточні центроїди, знаходить найближчий центроїд для кожного документа та повертає мітки кластерів, лічильники, частку inertia і часткові суми векторів за кластерами.
- **Reduce (master).** Master складає часткові суми від усіх воркерів, нормує їх і отримує нові центроїди; потім розсилає їх на наступній ітерації.
- **Асинхронність.** Виклики методів воркерів повертають promise, а блокування відбувається лише при зчитуванні значення (.value), тому всі воркери працюють одночасно.
- **Передача даних.** Чанк документів надсилається воркеру один раз (на першій ітерації) і кешується в його пам’яті. Далі між вузлами передаються лише центроїди (K × 1000 чисел) та часткові суми. Якщо воркер кеш втратив, він відповідає “MISS”, і master повторно надсилає чанк (у всіх експериментах таких випадків не було, cache_misses = 0). Дані передаються плоскими рядками, а числа записуються через repr(), тому відновлення точне.
- **Перевірка коректності.** Після паралельної версії master виконує повний обсяг обчислень послідовно (той самий код, один чанк, ті самі початкові центроїди) та порівнює мітки, кількість ітерацій й inertia.
- **Оцінка швидкодії.** Вимірюються: t_prep — підготовка TF-IDF; t_parallel — кодування чанків, їх передача, усі ітерації та редукція; t_serial — послідовна версія; окремо — час першої ітерації та середній час наступних.

3. Розгортання системи ПАРКС у Google Cloud Platform

Систему розгорнуто у проєкті Google Cloud (parks-510305), регіон Європа (Варшава), зона europe-central2-a. Створено п’ять віртуальних машин: один master і чотири воркери. Характеристики середовища наведено в таблиці 1, вигляд сторінки Nodes із чотирма зареєстрованими воркерами — на рисунку 1.

Таблиця 1 — Параметри тестового середовища

Параметр	Значення
Платформа	Google Compute Engine, europe-central2-a
Кількість вузлів	5 (1 master + 4 воркери)
Тип ВМ	n1-standard-1: 1 vCPU, 3.75 ГБ RAM (веб-інтерфейс ПАРКС показує 4 GB)
Процесор	Intel Xeon @ 2.20 GHz
ОС вузлів	Debian 12
Образ ПАРКС	hummer12007/parcs-node (Docker Hub); Python 2.7.13 у контейнері
Взаємодія вузлів	Pyro4 (RPC), внутрішня мережа VPC; веб-інтерфейс master — порт 8080
Керування кількістю воркерів	кнопки Disable / Enable на вкладці Nodes

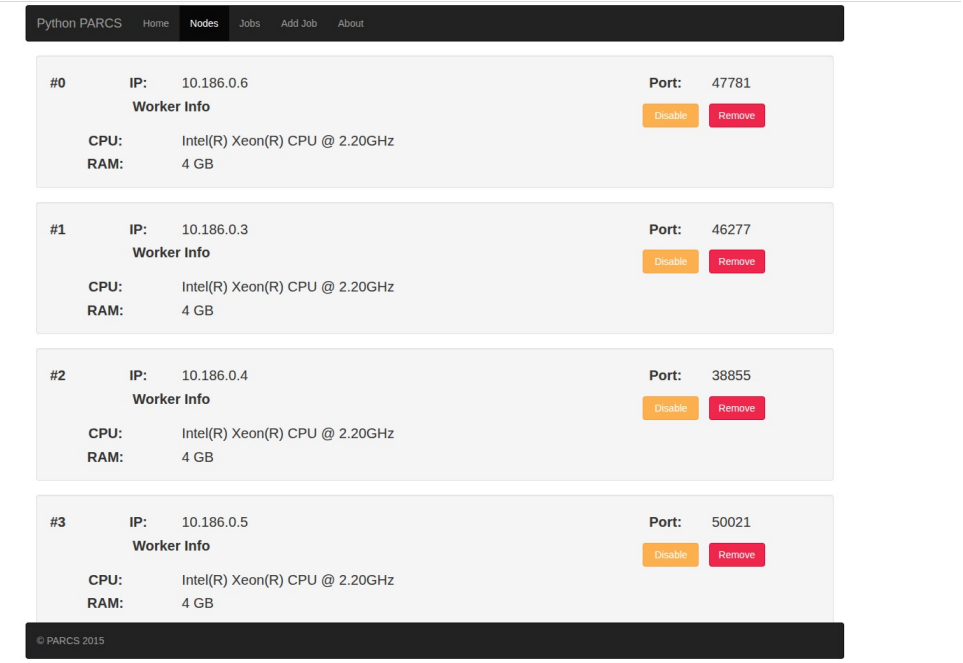


Рисунок 1 — Вкладка Nodes: чотири зареєстровані воркери (10.186.0.3–10.186.0.6)

Порядок розгортання (виконувався в Cloud Shell):

- створено проєкт, увімкнено оплату та Compute Engine API; задано зону europe-central2-a;
- створено правило фаєрвола allow-all (для навчальних цілей) для з’єднання вузлів і доступу до порту 8080;
- створено ВМ master із startup-скриптом, який встановлює Docker і запускає контейнер з PARCS_ARGS=“master”;
- створено ВМ worker1–worker4 з аналогічним скриптом і PARCS_ARGS=“worker <внутрішня IP master>”.

Під час розгортання виникли такі особливості, які відрізняються від базової інструкції:

- команда create-with-container (запуск контейнера під час створення ВМ) більше не підтримується Google Cloud, тому використано звичайні ВМ Debian зі startup-скриптом, що встановлює Docker і запускає контейнер з параметром --network=host;
- у зоні europe-north1-a тип n1-standard-1 був недоступний (ZONE_RESOURCE_POOL_EXHAUSTED), тому всі вузли створено в europe-central2-a;

- в образі працює Python 2.7, який відхиляє файл із нелатинськими символами без оголошення кодування (SyntaxError: Non-ASCII character), тому код розв’язку містить лише ASCII-символи й сумісний із Python 2.7 і 3.

4. Реалізація

Розв’язок оформлено як один файл (kmeans_solver.py) із класом Solver, як того вимагає PARCS-python: конструктор __init__(workers, input_file_name, output_file_name) і метод solve(). Метод assign_chunk позначено @expose, тому його можна викликати на воркері віддалено. Повний код наведено в додатку А. Використано лише стандартну бібліотеку Python. Ключовий фрагмент — функція Map на воркері:

```
@staticmethod
@expose
def assign_chunk(chunk, key, cent_str, k, V):
    """chunk: documents as a flat string (see encode_docs), or None if the chunk is
    already cached on the worker under key (otherwise returns "MISS").
    cent_str: k*V centroid values as a flat string.
    Returns [labels, counts, inertia, sums_string]."""
    if chunk is None:
        docs = Solver._cache.get(key)
        if docs is None:
            return "MISS"
    else:
        docs = Solver.decode_docs(chunk)
        Solver._cache[key] = docs
    flat = [float(x) for x in cent_str.split()]
    centroids = [flat[j * V:(j + 1) * V] for j in range(k)]
    cnorm = [sum(x * x for x in c) for c in centroids]
    sums = [[0.0] * V for _ in range(k)]
    counts = [0] * k
    labels = []
    inertia = 0.0
    for doc in docs:
        best, bd = 0, 1e18
        for j in range(k):
            c = centroids[j]
            dot = 0.0
            for i, v in doc:
                dot += v * c[i]
            d = 1.0 - 2.0 * dot + cnorm[j]      # ||x|| = 1
            if d < bd:
                bd, best = d, j
        labels.append(best)
        counts[best] += 1
        inertia += bd
        s = sums[best]
        for i, v in doc:
            s[i] += v
    return [labels, counts, inertia, Solver.encode_floats(sums)]
```

Параметри запуску (K, кількість ітерацій, tol, розмір словника, чи рахувати послідовну версію) можна задавати у першому рядку вхідного файлу, наприклад “#PARCS iters=10 tol=0 k=20”. Це дозволяє змінювати обсяг обчислень, не переписуючи код.

Оптимізація передачі даних

Перший прототип передавав документи вкладеними списками пар [індекс, значення] і працював із K = 8. На 16 655 документах і чотирьох воркерах він показав t_serial = 11.03 с проти t_parallel = 17.93 с, тобто прискорення 0.615 — паралельна версія виявилась

повільнішою. Причиною були витрати на серіалізацію даних. Локальний вимір (Python 3.11, серіалізатор serpent з Pyro4) показав, що чанк із 4163 документів у вигляді списку пар має розмір 11.3 МБ і потребує 1.2 с на серіалізацію та 4.3 с на десеріалізацію, тоді як плоский рядок займає 5.9 МБ і потребує 0.04 с та 0.07 с (плюс близько 0.7 с на розбір на воркері). Тому в підсумковій версії дані передаються плоскими рядками, чанк кешується на воркері, а число кластерів збільшено до K = 20 (відповідає 20 групам корпусу), що збільшило обсяг обчислень на одиницю переданих даних. Прототип і підсумкова версія тестувалися на різних наборах, тому їхні числа не слід порівнювати напряму; наведені дані ілюструють природу проблеми.

5. Дані та методика експерименту

Для експериментів використано корпус 20 Newsgroups (бібліотека scikit-learn, 20 тематичних груп новин; заголовки, підписи та цитати видалено, відібрано документи довші за 20 слів). Оригінальний корпус має близько 18 тис. документів, що замало для вимірювання масштабування, тому всі три тестові набори згенеровано режимом аугментації: кожен новий документ складається зі слів двох випадкових документів однієї групи (приблизно 60 % слів кожного, перемішаних, не більше 150 слів), а мітка групи зберігається. Розміри наборів — 5 000, 25 000 і 50 000 документів.

Параметри запуску однакові в усіх експериментах: K = 20 кластерів, словник 1000 слів, фіксовані 10 ітерацій (tol = 0), seed = 42. Кількість воркерів змінювали від 1 до 4 вимкненням зайвих вузлів у веб-інтерфейсі. Для кожної комбінації виконано один запуск (невдалий запуск через помилку веб-інтерфейсу було повторено; див. додаток Б).

Прискорення обчислювали двома способами: $S_{ser} = T_{послід} / T_{парал}$ — відносно послідовної версії, що виконується на master; $S_1 = T_{парал}(1\ воркер) / T_{парал}(p\ воркерів)$ — відносно тієї ж паралельної програми на одному воркері. Ефективність $E = S_1 / p$. Частку роботи, що не прискорюється, оцінено метрикою Карпа–Флатта $e = (1/S - 1/p) / (1 - 1/p)$.

6. Результати

Таблиця 2 — Час виконання та прискорення (K = 20, 10 ітерацій)

Документів	Воркерів	T_prep, с	T_послід., с	T_парал., с	S_ser	S_1	E
5 000	1	0.75	6.06	5.23	1.16	1.00	1.00
5 000	2	0.76	6.03	3.46	1.74	1.51	0.76
5 000	4	0.96	6.04	2.52	2.40	2.08	0.52
25 000	1	4.34	28.95	22.32	1.30	1.00	1.00
25 000	2	4.59	28.21	12.82	2.20	1.74	0.87
25 000	4	4.18	29.30	7.74	3.79	2.88	0.72
50 000	1	8.59	58.64	43.04	1.36	1.00	1.00
50 000	2	8.70	57.15	24.23	2.36	1.78	0.89
50 000	4	8.01	56.99	14.66	3.89	2.94	0.73

T_prep — підготовка TF-IDF (послідовна, на master); T_послід. — послідовна версія на master; T_парал. — паралельна частина (без T_prep).

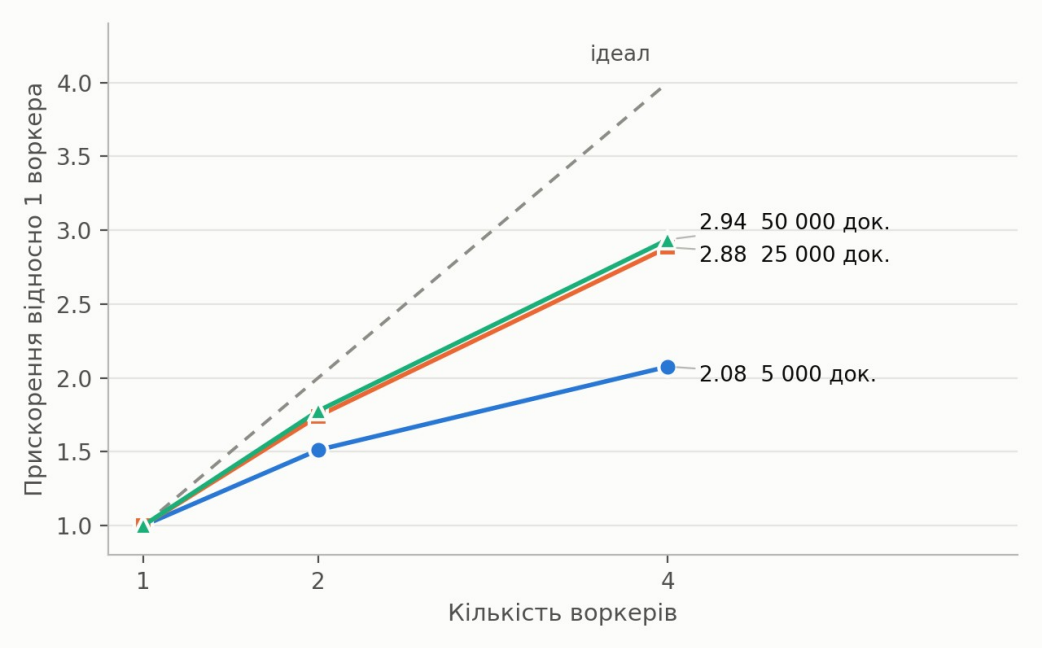


Рисунок 2 — Прискорення відносно паралельної програми на одному воркері

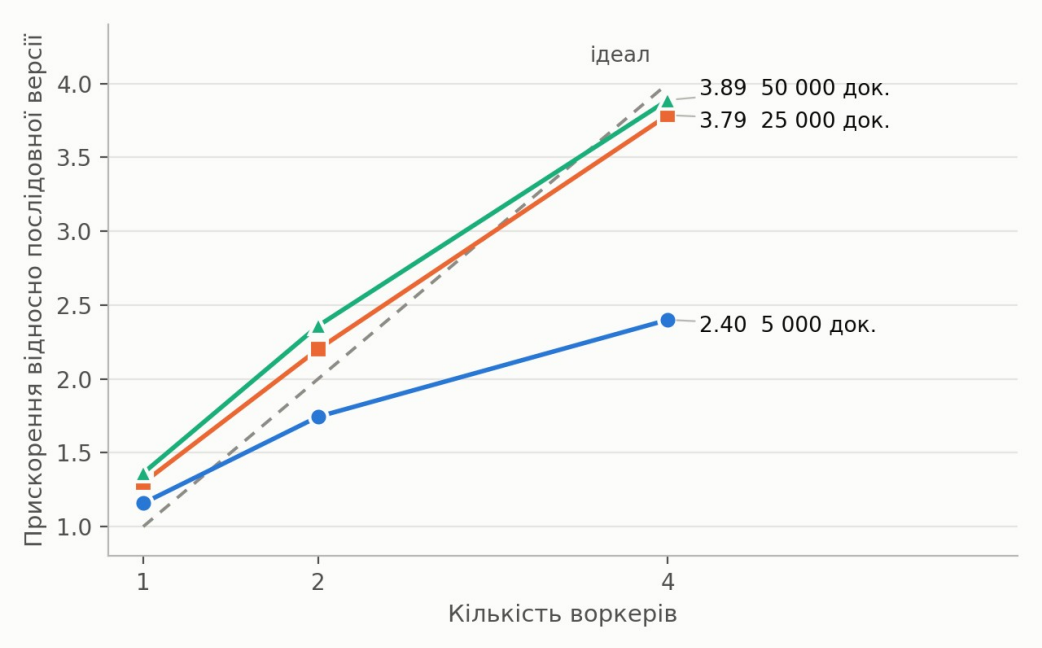


Рисунок 3 — Прискорення відносно послідовної версії на master

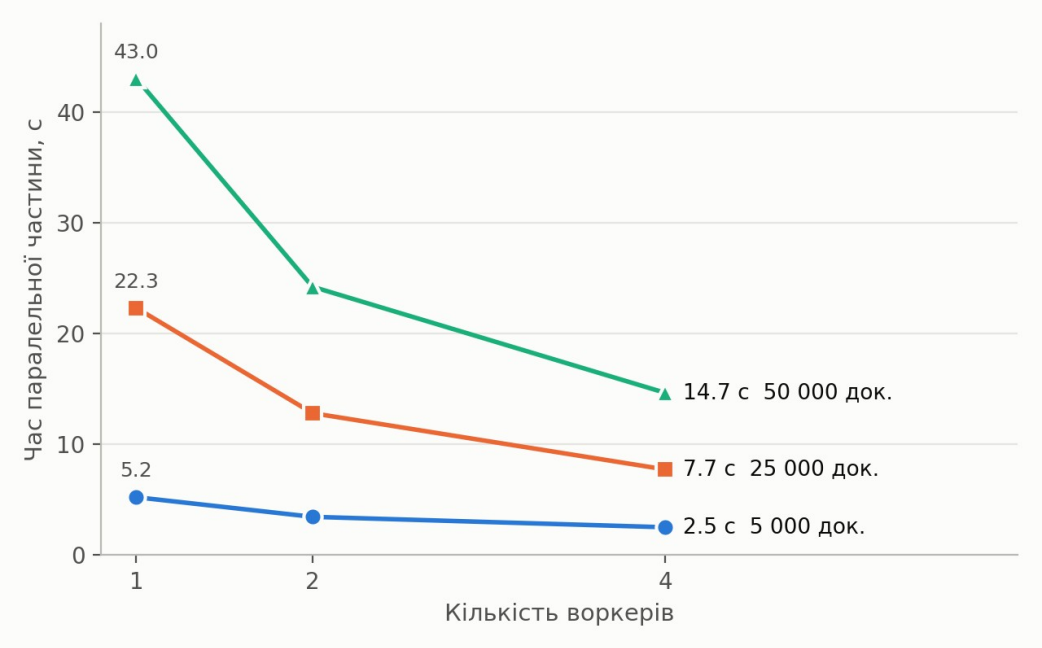


Рисунок 4 — Час паралельної частини залежно від кількості воркерів

Таблиця 3 — Структура часу паралельної частини та накладні витрати

Документів	Воркерів	1-ша ітерація, с	Наступні (середнє), с	Одноразова передача, с	Карп–Флатте	S_1 з T_prep
5 000	1	0.98	0.45	0.53	—	1.00
5 000	2	0.48	0.31	0.16	0.323	1.42
5 000	4	0.30	0.23	0.07	0.309	1.78
25 000	1	3.99	1.95	2.05	—	1.00
25 000	2	2.31	1.08	1.24	0.149	1.55
25 000	4	1.02	0.66	0.36	0.129	2.22
50 000	1	6.89	3.84	3.05	—	1.00
50 000	2	3.73	2.10	1.63	0.126	1.57
50 000	4	2.14	1.21	0.93	0.121	2.25

Одноразова передача = час першої ітерації мінус середній час наступних (передача чанка воркеру та його розбір). “S_1 з T_prep” — прискорення відносно 1 воркера з урахуванням послідовної підготовки TF-IDF.

7. Аналіз результатів

7.1. Масштабування

Час паралельної частини стабільно зменшується зі збільшенням кількості воркерів. Для 50 000 документів він становить 43.0 с на одному воркері, 24.2 с на двох і 14.7 с на чотирьох; прискорення відносно одного воркера — 1.78 і 2.94, ефективність — 0.89 і 0.73. Для 25 000 документів — 1.74 і 2.88 (ефективність 0.87 і 0.72). На малому наборі в 5 000 документів прискорення значно скромніше: 1.51 і 2.08 (ефективність 0.76 і 0.52). Ефективність зростає з обсягом даних, оскільки частка корисних обчислень порівняно з витратами на мережеву взаємодію збільшується — типова поведінка для задач із фіксованими накладними витратами на ітерацію.

7.2. Накладні витрати

Метрика Карпа–Флатта показує, яка частка роботи не прискорюється з ростом p: для 25 000 і 50 000 документів вона становить 0.12–0.15, а для 5 000 документів — 0.31–0.32. Джерела цих витрат — розсилання центроїдів, збирання й розбір часткових сум на master (редукція виконується послідовно), мережеві затримки Pyro4 та послідовне кодування чанків на master. Перша ітерація довша за наступні на величину одноразової передачі чанків (для 25 000 документів — 2.05 с на одному воркері, 0.36 с на чотирьох; для 50 000 — 3.05 і 0.93 с), після чого кешування чанка на воркері усуває цю вартість. Кеш не підводив жодного разу (cache_misses = 0).

7.3. Послідовна частина (закон Амдала)

Підготовка TF-IDF виконується послідовно на master (4.2 с для 25 000 і 8.0 с для 50 000 документів) і не входить у T_парал. З урахуванням цієї частини прискорення на чотирьох воркерах відносно одного становить 2.22 для 25 000 і 2.25 для 50 000 документів (таблиця 3), тобто послідовна підготовка помітно обмежує загальне прискорення. Її можна розпаралелити окремо (токенізація кожного документа незалежна), що є напрямом подальшого вдосконалення.

7.4. Особливість базової лінії

Прискорення відносно послідовної версії (S_ser) перевищує кількість воркерів вже для p = 1 і p = 2 (наприклад, 1.30 на одному воркері для 25 000 документів), тобто ефективність за цією метрикою більша за одиницю. Це не означає суперлінійного прискорення в звичайному сенсі: один воркер і master виконують один і той самий код на тому самому обсязі даних, але середній час ітерації на master на 45 % більший за 25 000 документів (2.81 с проти 1.95 с), на 49 % за 50 000 і на 30 % за 5 000 (рисунок 5). Імовірна причина — відмінності середовища: master одночасно обслуговує веб-інтерфейс і планувальник завдань та тримає в пам’яті весь корпус (тексти й вектори), тоді як воркер зберігає лише свій чанк, що може збільшувати накладні витрати керування пам’яттю в Python 2. Ця гіпотеза окремо не перевірялась. Тому коректнішою мірою масштабування вважається прискорення відносно одного воркера (S_1), а S_ser наведено для повноти й порівняння з послідовною версією.

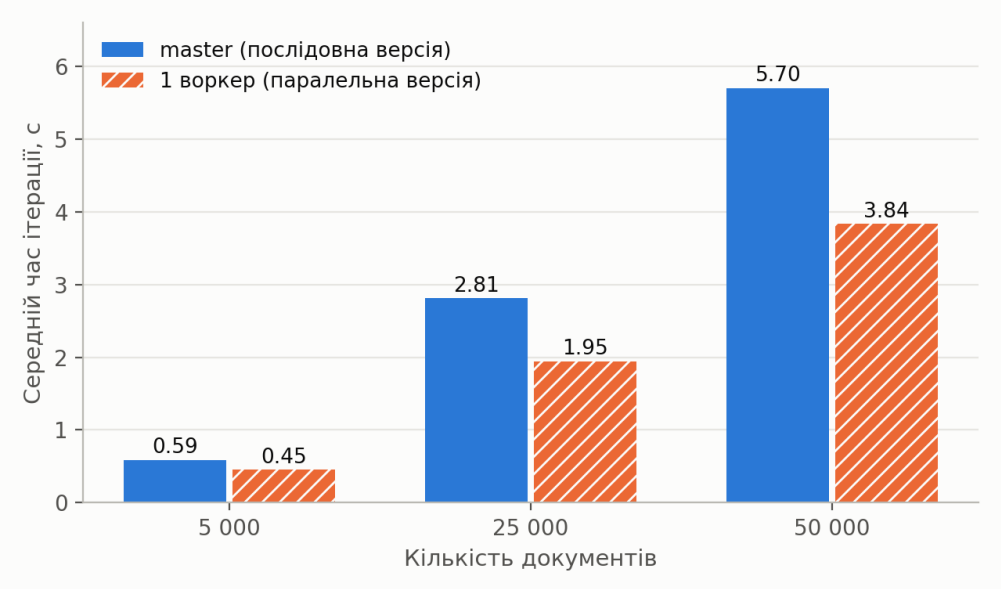


Рисунок 5 — Середній час ітерації: послідовна версія на master та паралельна версія на одному воркері

7.5. Збіг послідовного й паралельного результатів

У восьми з дев’яти запусків мітки кластерів паралельної та послідовної версій збіглися для кожного документа; значення inertia збігаються між запусками з різною кількістю воркерів (наприклад, 36382.617445 для 25 000 документів). Єдиний запуск із прапором serial_equals_parallel: False — 50 000 документів на чотирьох воркерах: у ньому відрізняються мітки 5 документів із 50 000 (0.01 %), а inertia збігається до шостого знака. Причина — неасоціативність додавання чисел із плаваючою комою: у паралельній версії часткові суми воркерів додаються в іншому порядку, ніж у послідовній, тож центроїди відрізняються в останніх розрядах, і кілька документів, що лежать майже на межі двох кластерів, можуть перейти до сусіднього. Це очікуваний ефект, а не помилка алгоритму. Реалізована перевірка є суворою (вимагає повного збігу міток), для практичного використання доцільніше допускати невеликий відсоток розбіжностей.

7.6. Точність вимірювань

Послідовна версія виконувалась у кожному запуску й давала майже однаковий час: 6.03–6.06 с (5 000), 28.2–29.3 с (25 000) і 57.0–58.6 с (50 000), тобто розкид не більший за ±2 %. Це дає орієнтир випадкової похибки вимірювань, хоча для кожної конфігурації паралельної версії виконано лише один запуск.

8. Якість кластеризації

Для наочної перевірки змістовності кластерів у таблиці 4 наведено 20 кластерів для набору з 25 000 документів (4 воркери) із їхніми розмірами та найвагомішими словами центроїдів. Інтерпретація тем виконана за ключовими словами й має суб’єктивний характер.

Таблиця 4 — Кластери для 25 000 документів (K = 20)

№	Документів	Ключові слова центроїда	Тема
0	965	sale, offer, shipping, condition, new, asking	Продаж, оголошення
1	1 583	game, team, games, hockey, play, players	Спорт (хокей, бейсбол)
2	1 107	windows, dos, file, files, any, using	MS Windows / DOS
3	821	key, clipper, chip, encryption, keys, government	Криптографія
4	1 483	god, jesus, bible, christian, who, his	Релігія (християнство)
5	860	problem, com, could, any, time, don	Загальна лексика (змішаний)
6	812	window, server, motif, display, application, running	X Window
7	1 692	use, software, used, any, get, like	Програмне забезпечення (змішаний)
8	1 050	card, monitor, apple, video, mac, mhz	Апаратне забезпечення (Mac, PC)
9	1 480	bike, back, like, off, good, get	Мотоцикли / велосипеди
10	1 393	his, him, who, don, time, did	Розмовна лексика (змішаний)
11	621	israel, jews, jewish, who, peace, people	Близький Схід
12	1 938	don, just, why, think, like, know	Загальна лексика (змішаний)
13	931	car, cars, engine, dealer, any, just	Автомобілі
14	1 584	thanks, anyone, any, please, know, does	Запити про допомогу
15	2 551	people, who, such, see, any, say	Дискусійна лексика (змішаний)
16	933	drive, scsi, disk, ide, drives, hard	Накопичувачі, SCSI
17	785	space, image, nasa, program, images, earth	Космос, зображення
18	1 762	government, gun, people, law, who, fbi	Політика, зброя, право
19	649	she, after, who, know, did, just	Розмовна лексика (змішаний)

Значна частина кластерів відповідає окремим темам початкових груп новин (спорт, криптографія, релігія, автомобілі, космос, накопичувачі, X Window, Близький Схід). Частина кластерів (№ 5, 7, 10, 12, 15, 19) є змішаними й складається із загальновживаної розмовної лексики; це типова особливість TF-IDF-кластеризації без глибшого очищення тексту й без зменшення розмірності. Кількісні міри збігу з відомими групами (purity, NMI) можна розрахувати скриптом evaluate.py за збереженими мітками груп.

Висновок

У роботі реалізовано послідовний і паралельний алгоритми кластеризації текстових документів (TF-IDF та сферичний K-Means) для системи ПАРКС (PARCS-python), систему розгорнуто в Google Cloud Platform (master і чотири воркери), проведено експерименти з 1, 2 і 4 воркерами на наборах із 5 000, 25 000 і 50 000 документів. Коректність підтверджено: у восьми з дев’яти запусків результати паралельної та послідовної версій збіглися повністю, в одному — відрізнялися мітки 5 документів із 50 000 через порядок додавання чисел із плаваючою комою.

Паралельна версія масштабується тим краще, чим більший обсяг даних. На 50 000 документів чотири воркери прискорюють паралельну частину в 2.94 раза відносно одного воркера (ефективність 0.73) і в 3.89 раза відносно послідовної версії на master; на 5 000 документів ефективність нижча (0.52), бо накладні витрати на мережеву взаємодію відносно більші. Основні обмеження масштабування — послідовна підготовка TF-IDF на master, одноразова передача чанків та послідовна редукція часткових сум. Експеримент також показав, що спосіб передачі даних має вирішальний вплив: перехід від вкладених списків до плоских рядків із кешуванням чанка на воркері перетворив паралельну програму з повільнішої за послідовну на таку, що дає помітне прискорення.

Подальші напрями вдосконалення: розпаралелити токенізацію та побудову TF-IDF, виконувати редукцію деревом, виміряти масштабування на більшій кількості воркерів і повторити вимірювання кілька разів для оцінки розкиду. Після завершення експериментів віртуальні машини слід видалити, а правило фаєрвола allow-all — вимкнути.

Додаток А. Код розв’язку (kmeans_solver.py)

```
# -*- coding: utf-8 -*-
"""
PARCS-python: parallel text clustering (TF-IDF + spherical K-Means).

Scheme (Map-Reduce on every iteration):
  master: tokenization -> TF-IDF -> initial centroids
  loop:   MAP    - a worker gets its chunk of documents (sent once, cached on the
                   worker) + the current centroids, returns labels, counts,
                   inertia and partial per-cluster sums
          REDUCE - master adds the partial sums, normalizes them, gets new centroids
Compatible with Python 2.7 and 3.x, standard library only. ASCII only.

All bulk data is passed between nodes as flat strings (fast to (de)serialize with
Pyro4/serpent); floats are written with repr(), so the round trip is exact and the
serial and parallel runs give identical results.
"""
import io
import math
import random
import re
import time

try:
    from Pyro4 import expose
except ImportError:          # local run without Pyro4
    def expose(f):
        return f

K = 20               # number of clusters
MAX_ITER = 15        # max iterations
MAX_FEATURES = 1000  # vocabulary size
TOL = 1e-3           # relative inertia change for stopping
SEED = 42
RUN_SERIAL = True    # also run the serial version on master for comparison

STOP = set(("the and for are but not you all can had her was one our out has have this "
            "that with from they will would there their what about which when your said "
            "them these some into than then its also been were more other only").split())


class Solver:
    _cache = {}   # decoded chunks cached on the worker

    def __init__(self, workers=None, input_file_name=None, output_file_name=None):
        self.input_file_name = input_file_name
        self.output_file_name = output_file_name
        self.workers = workers
        print("Inited")

    # ------------------------------------------------------------------- master
    def solve(self):
        global K, MAX_ITER, TOL, MAX_FEATURES, RUN_SERIAL
        print("Job Started")
        W = len(self.workers)
        print("Workers %d" % W)

        docs, cfg = self.read_input()
        # optional first line of the input file: "#PARCS k=20 iters=10 tol=0 features=1000 serial=1"
        if "k" in cfg:
            K = int(cfg["k"])
        if "iters" in cfg:
            MAX_ITER = int(cfg["iters"])
        if "tol" in cfg:
            TOL = float(cfg["tol"])
```

```
        if "features" in cfg:
            MAX_FEATURES = int(cfg["features"])
        if "serial" in cfg:
            RUN_SERIAL = bool(int(cfg["serial"]))
        print("Config: K=%d iters=%d tol=%g features=%d serial=%s" % (K, MAX_ITER, TOL, MAX_FEATURES,
RUN_SERIAL))

        t0 = time.time()
        vecs, vocab = Solver.build_tfidf(docs, MAX_FEATURES)
        t_prep = time.time() - t0
        n, V = len(vecs), len(vocab)
        init = Solver.init_centroids(vecs, K, V, SEED)
        print("Docs %d, vocab %d, preprocessing %.3f s" % (n, V, t_prep))

        # ---- parallel version (chunk encoding and the first transfer are included in the time)
        workers = self.workers
        token = "%d_%d" % (int(time.time() * 1000), random.randrange(10 ** 9))
        stats = {"miss": 0}
        t0 = time.time()
        step = (n + W - 1) // W
        chunk_str = [Solver.encode_docs(vecs[i * step:(i + 1) * step]) for i in range(W)]
        sent = [False] * W
        nocache = [False] * W

        def par_parts(centroids):
            cstr = Solver.encode_floats(centroids)   # encoded once, shared by all workers
            promises = []
            for w in range(W):
                payload = chunk_str[w] if (nocache[w] or not sent[w]) else None
                promises.append(workers[w].assign_chunk(payload, token + "_%d" % w, cstr, K, V))
            res = [p.value for p in promises]
            for w in range(W):
                if res[w] == "MISS":              # worker lost its cache -> resend and stop relying on it
                    stats["miss"] += 1
                    nocache[w] = True
                    res[w] = workers[w].assign_chunk(chunk_str[w], token + "_%d" % w, cstr, K, V).value
                sent[w] = True
            return res

        lab_p, inr_p, it_p, cent_p, cnt_p, times_p = Solver.kmeans_loop(par_parts, init, V)
        t_par = time.time() - t0
        print("Parallel done: %.3f s, %d iterations" % (t_par, it_p))

        # ---- serial version (for correctness check and speedup); same code path, one chunk
        t_ser = None
        match = None
        times_s = []
        if RUN_SERIAL:
            t0 = time.time()
            whole = Solver.encode_docs(vecs)

            def ser_parts(centroids):
                cstr = Solver.encode_floats(centroids)
                return [Solver.assign_chunk(whole, "serial_" + token, cstr, K, V)]

            lab_s, inr_s, it_s, _, _, times_s = Solver.kmeans_loop(ser_parts, init, V)
            t_ser = time.time() - t0
            Solver._cache.pop("serial_" + token, None)
            same = sum(1 for a, b in zip(lab_p, lab_s) if a == b)
            match = (same == n and it_s == it_p and abs(inr_s - inr_p) <= 1e-6 * max(1.0, inr_s))
            print("Serial done: %.3f s, labels equal: %s" % (t_ser, match))

        self.write_output(n, V, W, it_p, t_prep, t_par, t_ser, inr_p, match, cnt_p, cent_p, vocab,
                          lab_p, times_p, times_s, stats["miss"], sum(len(c) for c in chunk_str))

    # ------------------------------------------------------------------ map (worker)
```

```
@staticmethod
@expose
def assign_chunk(chunk, key, cent_str, k, V):
    """chunk: documents as a flat string (see encode_docs), or None if the chunk is
    already cached on the worker under key (otherwise returns "MISS").
    cent_str: k*V centroid values as a flat string.
    Returns [labels, counts, inertia, sums_string]."""
    if chunk is None:
        docs = Solver._cache.get(key)
        if docs is None:
            return "MISS"
    else:
        docs = Solver.decode_docs(chunk)
        Solver._cache[key] = docs
    flat = [float(x) for x in cent_str.split()]
    centroids = [flat[j * V:(j + 1) * V] for j in range(k)]
    cnorm = [sum(x * x for x in c) for c in centroids]
    sums = [[0.0] * V for _ in range(k)]
    counts = [0] * k
    labels = []
    inertia = 0.0
    for doc in docs:
        best, bd = 0, 1e18
        for j in range(k):
            c = centroids[j]
            dot = 0.0
            for i, v in doc:
                dot += v * c[i]
            d = 1.0 - 2.0 * dot + cnorm[j]      # ||x|| = 1
            if d < bd:
                bd, best = d, j
        labels.append(best)
        counts[best] += 1
        inertia += bd
        s = sums[best]
        for i, v in doc:
            s[i] += v
    return [labels, counts, inertia, Solver.encode_floats(sums)]

# ------------------------------------------------------------------- reduce (master)
@staticmethod
def reduce_parts(parts, old, k, V):
    sums = [[0.0] * V for _ in range(k)]
    counts = [0] * k
    inertia = 0.0
    labels = []
    for lab, cnt, inr, sstr in parts:
        labels.extend(lab)
        inertia += inr
        flat = [float(x) for x in sstr.split()]
        for j in range(k):
            counts[j] += cnt[j]
            row = sums[j]
            base = j * V
            for i in range(V):
                row[i] += flat[base + i]
    new = []
    for j in range(k):
        if counts[j] == 0:
            new.append(old[j])
            continue
        norm = math.sqrt(sum(x * x for x in sums[j])) or 1.0
        new.append([x / norm for x in sums[j]])
    return new, counts, inertia, labels

@staticmethod
```

```
    def kmeans_loop(compute_parts, init, V):
        centroids, prev = init, None
        labels, counts, inertia, it = [], [], 0.0, 0
        times = []
        for it in range(1, MAX_ITER + 1):
            t = time.time()
            parts = compute_parts(centroids)
            centroids, counts, inertia, labels = Solver.reduce_parts(parts, centroids, K, V)
            times.append(time.time() - t)
            if TOL > 0 and prev is not None and abs(prev - inertia) <= TOL * prev:
                break
            prev = inertia
        return labels, inertia, it, centroids, counts, times

    # ------------------------------------------------------------------ transport helpers
    @staticmethod
    def encode_floats(rows):
        return " ".join(" ".join(repr(x) for x in row) for row in rows)

    @staticmethod
    def encode_docs(docs):
        return "\n".join(" ".join("%d:%s" % (i, repr(v)) for i, v in d) for d in docs)

    @staticmethod
    def decode_docs(text):
        out = []
        for line in text.split("\n"):
            d = []
            for tok in line.split():
                a, b = tok.split(":")
                d.append((int(a), float(b)))
            out.append(d)
        return out

    # ------------------------------------------------------------------ NLP
    @staticmethod
    def tokenize(text):
        return [w for w in re.findall(r"[a-z]{3,}", text.lower()) if w not in STOP]

    @staticmethod
    def build_tfidf(docs, max_features):
        n = len(docs)
        toks = [Solver.tokenize(d) for d in docs]
        df = {}
        for t in toks:
            for w in set(t):
                df[w] = df.get(w, 0) + 1
        cand = [(c, w) for w, c in df.items() if c >= 2 and c <= 0.5 * n]
        cand.sort(key=lambda x: (-x[0], x[1]))
        vocab = [w for _, w in cand[:max_features]]
        index = dict((w, i) for i, w in enumerate(vocab))
        idf = [math.log((1.0 + n) / (1.0 + df[w])) + 1.0 for w in vocab]
        vecs = []
        for t in toks:
            tf = {}
            for w in t:
                i = index.get(w)
                if i is not None:
                    tf[i] = tf.get(i, 0) + 1
            vec = sorted([i, (1.0 + math.log(c)) * idf[i]] for i, c in tf.items())
            norm = math.sqrt(sum(v * v for _, v in vec)) or 1.0
            # values are rounded to 6 significant digits: smaller transfer, identical in serial and
parallel
            vecs.append([(i, float("%.6g" % (v / norm))) for i, v in vec])
        return vecs, vocab
```

```
@staticmethod
def init_centroids(vecs, k, V, seed):
    rnd = random.Random(seed)
    cents = []
    for idx in rnd.sample(range(len(vecs)), k):
        row = [0.0] * V
        for i, v in vecs[idx]:
            row[i] = v
        cents.append(row)
    return cents

# ---------------------------------------------------------------- I/O
def read_input(self):
    cfg = {}
    docs = []
    with io.open(self.input_file_name, 'r', encoding='utf-8', errors='ignore') as f:
        for n, line in enumerate(f):
            line = line.strip()
            if n == 0 and line.startswith("#PARCS"):
                for tok in line.split()[1:]:
                    if "=" in tok:
                        a, b = tok.split("=", 1)
                        cfg[a] = b
                continue
            if line:
                docs.append(line)
    return docs, cfg

def write_output(self, n, V, W, iters, t_prep, t_par, t_ser, inertia, match, counts, cents, vocab,
                 labels, times_p, times_s, misses, sent_chars):
    f = open(self.output_file_name, 'w')
    f.write("docs: %d\nvocab: %d\nclusters: %d\nworkers: %d\niterations: %d\n" % (n, V, K, W, iters))
    f.write("t_prep: %.4f\nt_parallel: %.4f\n" % (t_prep, t_par))
    if t_ser is not None:
        f.write("t_serial: %.4f\nspeedup: %.3f\nefficiency: %.3f\nserial_equals_parallel: %s\n"
                % (t_ser, t_ser / t_par, t_ser / t_par / W, match))
    f.write("par_iter_first: %.4f\n" % times_p[0])
    if len(times_p) > 1:
        f.write("par_iter_avg_rest: %.4f\n" % (sum(times_p[1:]) / (len(times_p) - 1)))
    if times_s:
        f.write("ser_iter_avg: %.4f\n" % (sum(times_s) / len(times_s)))
    f.write("cache_misses: %d\nchunk_chars_sent: %d\n" % (misses, sent_chars))
    f.write("inertia: %.6f\ncluster_sizes: %s\n" % (inertia, " ".join(str(c) for c in counts)))
    for j in range(K):
        top = sorted(range(V), key=lambda i: -cents[j][i])[:8]
        f.write("cluster %d (%d): %s\n" % (j, counts[j], ", ".join(vocab[i] for i in top)))
    f.write("LABELS: %s\n" % " ".join(str(x) for x in labels))
    f.close()
    print("output done")
```

Додаток Б. Журнал завдань на сервері ПАРКС

Нижче наведено завдання з вкладки Jobs. Час початку — за UTC (за київським часом на три години пізніше). Тривалість включає також послідовну версію, яку master виконує для порівняння, тому для оцінки прискорення використовуються значення з вихідних файлів (таблиця 2), а не ця тривалість.

№	Назва завдання	Початок (UTC)	Тривалість	Примітка
0	KMEANS_W4_TEST	06:42:55	0:0:0	Невдалий запуск: у коді були кириличні коментарі (Python 2.7)
1	KMEANS_W4_TEST	07:00:30	0:0:33	Прототип (16 655 док., K = 8, 4 воркери), прискорення 0.615
2	KM_25K_W4	08:40:05	0:0:42	25 000 док., 4 воркери
3	OUT_50K_W4.TXT	08:58:30	0:1:21	50 000 док., 4 воркери
4	OUT_5K_W4.TXT	09:02:41	0:0:10	5 000 док., 4 воркери
5	OUT_25K_W2.TXT	09:43:51	0:0:46	25 000 док., 2 воркери
6	OUT_50K_W2.TXT	09:45:51	0:1:31	50 000 док., 2 воркери
7	OUT_5K_W2.TXT	09:48:32	0:0:11	5 000 док., 2 воркери
8	OUT_25K_W1.TXT	09:51:47	0:0:56	25 000 док., 1 воркер
9	OUT_50K_W1.TXT	09:53:52	0:0:0	Невдалий запуск (без повідомлення про причину), перезапущено як № 10
10	OUT_50K_W1.TXT	09:55:55	0:1:51	50 000 док., 1 воркер
11	OUT_5K_W1.TXT	09:58:41	0:0:13	5 000 док., 1 воркер